

M2-1: Data Documentation

Group 6

24 October 2025

Group 6 Members						
Name	Omar Elfiki	Alexander Payn	Ashour Kariakous	Long Nguyen	Lorenzo Franca	Alexandru Mihăilă
Student ID	i6344736	i6385871	i6385235	i6389825	i6391129	i6400384

Research Question

Can we predict the time-to-threshold, denoting number of training steps and wall-clock time required for an agent to reach a performance threshold using initial configuration and early learning-curve signals?

Targets (to predict)

- `wall_clock_time` (s) (observed seconds).
- *Time-to-threshold (derived)* from per-step reward logs: steps/time to reach a target reward

Data Collection Process

The data collection process is automated through a Python script that wraps around the Unity ML-Agents toolkit. Each training run is initiated using the script `train_model.py`, which launches the training run through the ML-Agents interface and records both training and system performance data.

The script performs three main functions:

1. **Training Execution:** The script starts a new training session by calling the ML-Agents CLI with a specified configuration file (`.yaml`) and a unique run identifier. This ensures each experiment is fully reproducible and associated with its training parameters.
2. **System Monitoring:** During training, a background monitoring thread records the system's **CPU usage**, **RAM usage**, and total **execution time**. This allows performance evaluation of different configurations under various hardware loads.
3. **Result Extraction:** After training completes, the script automatically parses the TensorBoard event logs produced by ML-Agents. It extracts relevant metrics such as:
 - Mean Policy Reward
 - Mean Policy Loss
 - Mean Value Loss
 - Mean Entropy

These are computed over the first fixed window of training steps (e.x. the first 24k steps after data appears).

The script also reads the generated `configuration.yaml` file from the results directory to capture the effective training parameters (e.g., algorithm, batch size, learning rate, and total steps).

All collected information, including hardware statistics, training parameters, and averaged performance metrics, is compiled into a structured data entry. Each record is appended automatically to two centralized files stored under the `/data` directory:

- `combined_results.csv`
- `combined_results.json`

This approach ensures that every run is consistently logged, making it possible to perform analysis on model performance, training efficiency, and resource utilization.

Data Integrity

By using a standardized naming conventions and git contribution rules, we can minimize the risk of data loss or disorganization. Each training run carries initials which can be referenced to obtain system hardware information such as CPU, RAM, and OS information.

We also plan to use seeding while gathering data to insure experiment replicability from one system to another.

Relation to Research Question.

The collected data directly supports our research question, which investigates whether it is possible to *predict the time-to-threshold* (the number of training steps and wall-clock time required for an agent to reach a given performance level) based on initial configuration parameters and early training signals.

By systematically logging both configuration-level features (e.x. batch size, learning rate, and algorithm type) and dynamic indicators from the first phase of training (such as cumulative reward, policy loss, and entropy), the resulting dataset provides a foundation for constructing predictive machine learning models. These models' goal is to estimate how quickly a given configuration will reach to the target performance threshold, enabling more efficient experimentation and resource allocation.

Column Dictionary

Column	Type	Description
run_id	string	Unique identifier for a run
behavior	string	Unity environment (e.x. 3DBall, Basic, Sorter)
seed	int	Random seed for reproducibility (-1 indicates default/random)
num_agents	int	Number of simultaneous agents/instances
algorithm	string	DRL algorithm (e.x. PP0)
steps	int	Completed total training steps
batch_size	int	Minibatch size per update
buffer_size	int	Experience buffer size
learning_rate	float	Optimizer learning rate
epochs	int	Epochs per update (if applicable)
cpu_usage_mean	float	Average CPU utilization (0-1)
ram_usage_mean	float	Average RAM utilization (0-1)
wall_clock_time (s)	float	Wall-clock time in seconds
step_average	int	Step interval used for computing running means
mean_policy_reward	float	Average policy reward
policy_reward_start_step	int	Step index where reward aggregation starts
mean_policy_loss	float	Average policy loss
policy_loss_start_step	int	Step index where policy-loss aggregation starts
mean_value_loss	float	Average value loss
value_loss_start_step	int	Step index where value-loss aggregation starts
mean_entropy	float	Average policy entropy
entropy_start_step	int	Step index where entropy aggregation starts.